



CAP Plugin Development Workshop

Emoji Annotation Plugin

Date: January 20, 2026

Contact: *fill out*

Version: 1.0

Table of Contents

1. CAP Plugin Development	3
1.1. Objective	3
1.2. Prerequisites	3
1.3. Task Overview	3
2. Step-by-Step Instructions	4
2.1. Project Initialization	4
2.2. Test Environment Setup	4
2.3. Plugin Foundation	5
2.4. CDS Model Structure Analysis	5
2.5. Plugin Implementation	6
2.6. cds add	6
2.7. cds build	6
2.8. Additional tasks	6
3. Additional Resources	7

1. CAP Plugin Development

1.1. Objective

This exercise guides you through the development of a custom SAP Cloud Application Programming Model (CAP) plugin that adds emoji support via entity annotations. You will learn the fundamental concepts of CAP plugin architecture, lifecycle hooks, and extensibility mechanisms.

1.2. Prerequisites

- Node.js
- SAP CAP SDK (@sap/cds-dk installed globally)
- Basic understanding of JavaScript and CAP framework
- Familiarity with CDS modeling language
- Completed all of [Getting Started](#) in Capire

1.3. Task Overview

You will create a reusable CAP plugin that automatically processes emoji annotations in CDS models during runtime. This plugin demonstrates the basic structure of CAP Plugins, how to read the CDS model structure and dynamically register handlers from that.

2. Step-by-Step Instructions

Development of CAP Plugins typically work in a way where the repository of the plugin includes a sample application that demonstrates how the plugin is used. In this sample, we use a *npm workspace* setup to use the plugin. That same application is typically also used to test the functionality of the plugin via integration tests.

2.1. Project Initialization

1. Create an empty directory for the plugin:

```
mkdir emoji-plugin  
cd emoji-plugin
```

2. Initialize the Node.js project:

```
npm init -y
```

3. Configure peer dependencies in `package.json`:

```
"peerDependencies": {  
  "@sap/cds": ">=9"  
}
```

2.2. Test Environment Setup

We will create a sample CAP application that can be used to develop, use and test the plugin.

1. Create a test directory structure:

```
mkdir -p tests/bookshop  
cd tests/bookshop
```

2. Initialize a sample CAP project:

```
cds init . --add sample
```

3. Remove all dependencies from `tests/bookshop/package.json` to ensure the plugin is tested in isolation.
4. Configure workspace in root `package.json`:

```
"workspaces": [
  "tests/bookshop"
]
```

5. Add the plugin as a dependency in `tests/bookshop/package.json`. It is important that the dependency here is the same as the name configured in the root level `package.json`:

```
"dependencies": {
  "emoji-plugin": "file:../../.."
}
```

2.3. Plugin Foundation

1. Create `cds-plugin.js` in the project root with the following structure:

```
const cds = require("@sap/cds")

console.log("1st - Plugin loaded")

cds.once('served', () => {
  console.log("2nd - Services served")
})
```

2. Do a root level `npm i`
3. `cd tests/bookshop`
4. `DEBUG=plugins cds watch`
5. Observe the log and understand when *1st* and *2nd* are triggered

In your output, you should now see that the emoji-plugin is loaded.

2.4. CDS Model Structure Analysis

1. Examine the structure of CDS models in your test project
2. Add a custom emoji annotation to an entity in `tests/bookshop/db/schema.cds`:

```
entity Books {
  title : String @emoji;
}
```

3. Understand that annotations follow a hierarchical structure in the CDS model
4. During runtime, access the CSN (Core Schema Notation) representation to inspect annotations

2.5. Plugin Implementation

Implement the core functionality in `cds-plugin.js`. For that, we can loop through the CSN (Core Schema Notation, which is the representation of the CDS model during runtime) and search for the `@emoji` annotation. Implement the handler that adds an emoji to the element that is annotated.

```
const cds = require("@sap/cds")

cds.once('served', (csn) => {
  for (const srv of cds.services) {
    if(srv.name === "db") continue
    for (const entity of srv.entities) {
      for (const element of entity.elements) {
        if (element["@emoji"]) {
          // Process the emoji annotation
        }
      }
    }
  }
})
```

Make sure the appended emoji is added in your local application.

2.6. cds add

Use the documentation to figure out how `cds add` works. When `cds add emoji` is executed, log something to the console.

2.7. cds build

Use the documentation to figure out how `cds build` works. When `cds build` is executed, log something to the console.

2.8. Additional tasks

When there is still time left, come up with new ideas of what the plugin can do and implement that or continue to the next section with additional resources.

3. Additional Resources

Key Concepts of CAP:

- [Concepts](#)
- [Features](#)

Implementation samples and guides:

- [Playbook](#) - at least you should have seen everything there
- [cds facade](#) - a good generic read to know what is available

Videos on CAP Development:

- [Best practices](#)
- [Less code, same logic](#) - starts to hint to plugin development when the main goal is to reduce custom code

Plugin specific documentation:

- [Node.js Plugins](#)
- [Java Plugins](#)
- [Design Time APIs](#) - add and build steps

Generic resources:

- [CAP Resources Page](#)